Medical Insurance Cost Prediction Using Supervised Machine Learning



Self-Certificate

This is to certify that the dissertation/project report entitled "**Medical Insurance Cost Prediction Using Supervised Machine Learning**" has been carried out solely by me as an original and authentic piece of work for the partial fulfilment of the requirements for the award of the Bachelor of Technology (B. Tech) degree in Data Science (DS) under the guidance of Raihan Mistry Sir.

A Project Report

Submitted By

Name: Krisnendu Das, Harsh Bhardwaj, Ritwik Kumar, Palak Sinha.

Acknowledgement

It gives me great satisfaction to present my solo project titled **“MEDICAL INSURANCE COST PREDICTION USING SUPERVISED MACHINE LEARNING”**, completed as a part of the academic requirements for the Bachelor of Technology degree.

First and foremost, I would like to convey my sincere appreciation to my project mentor, **Mr. Raihan Mistry Sir**, for his valuable guidance, constructive feedback, and continuous support during every stage of this project. His suggestions and encouragement have played an important role in helping me understand and implement the concepts of supervised machine learning.

I would also like to express my deepest gratitude to my parents, whose constant encouragement, patience, and trust have motivated me to overcome challenges and complete this work successfully.

My sincere thanks go to all my respected faculty members and friends for their valuable advice, cooperation, and encouragement throughout the project. Their support has contributed significantly to making this learning experience meaningful.

Lastly, I am grateful to everyone who directly or indirectly supported me during the completion of this project. Their encouragement has been a source of strength and motivation throughout this journey.

INDEX

SL No	Content	Page No
1	Scope of the project	5
2	Abstract	5 - 6
3	Introduction	6
4	Dataset & Importing Libraries	6 - 7
5	Exploratory Data Analysis	8
6	Data Preprocessing & Feature Engineering	11
7	Model training and Evaluation	12
8	Final Model selection and Extraction	14
9	Web Application & Interface	15
10	Deployment	16
11	Conclusion	18

SCOPE OF THE PROJECT

Medical Insurance Cost Prediction:

- Develop a machine learning model to estimate medical insurance charges based on age, gender, BMI, number of children, smoking habits, and region.
- Perform data preprocessing and feature engineering to prepare the dataset for model training.

Regression Model Implementation:

- Implement five regression algorithms: Linear Regression, Ridge Regression, Lasso Regression, Decision Tree Regression, and Random Forest Regression.
- Apply hyperparameter tuning to improve model performance.

Model Evaluation:

- Compare model performance using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R-squared (R^2).
- Select the most suitable model based on evaluation results.

Real-Time Prediction Interface:

- Develop a simple Streamlit web application where users enter their personal details and receive an estimated medical insurance cost.

Machine Learning Integration:

- Use numerical scaling and categorical encoding through a preprocessing pipeline.
- Save the trained model using Joblib for integration into the web application.

Deployment Ready:

- Organize the project with separate folders for datasets, notebooks, trained models, and application files.
- Prepare the Streamlit application for online deployment.

ABSTRACT

Medical insurance costs vary from person to person depending on several factors, such as age, gender, BMI, smoking habits, number of children, and geographical region. Estimating these expenses can be useful for understanding the factors that influence insurance charges. This project presents a medical insurance cost prediction system using supervised machine learning techniques.

The project uses the Medical Cost Personal Dataset, which contains information about individuals and their corresponding medical insurance charges. Data preprocessing and feature engineering are performed to prepare the dataset for model training. Five regression algorithms—Linear Regression, Ridge Regression, Lasso Regression, Decision Tree Regression, and Random Forest Regression—are implemented and compared using MAE, RMSE, and R^2 evaluation metrics.

The best-performing model is selected after evaluation and hyperparameter tuning. The trained model is integrated into a Streamlit web application where users can enter their personal details and receive an estimated medical insurance cost.

The project demonstrates how supervised machine learning can be used to analyze personal factors and predict medical insurance expenses through a simple and interactive application.

INTRODUCTION

In today's healthcare and insurance sector, estimating medical insurance expenses is an important task for both individuals and insurance providers. Insurance charges can vary significantly depending on several factors, including age, gender, body mass index (BMI), smoking habits, number of children, and geographical location. Understanding the relationship between these factors and insurance costs can help in making better financial decisions.

The **Medical Insurance Cost Prediction Using Supervised Machine Learning** project applies regression techniques to estimate an individual's insurance charges based on personal and lifestyle-related information. The dataset is explored and processed to identify important patterns and prepare the data for model training.

The project implements five machine learning regression algorithms:

- Linear Regression
- Ridge Regression
- Lasso Regression
- Decision Tree Regression
- Random Forest Regression

The models are evaluated using three performance metrics:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R-squared (R^2)

Hyperparameter tuning is also performed to improve model performance and identify the most suitable prediction model.

The main technologies and components used in this project are:

- **Python** for programming and model development.
- **Pandas and NumPy** for data loading and manipulation.
- **Matplotlib and Seaborn** for data visualization and analysis.
- **Scikit-learn** for preprocessing, model training, and evaluation.
- **Joblib** for saving the trained machine learning model.
- **Streamlit** for developing the interactive web application.

DATA SET & IMPORTING LIBRARIES

Dataset Description :

The dataset used in this project is the Medical Cost Personal Dataset, which contains information about individuals and their corresponding medical insurance charges. It consists of numerical and categorical attributes that help analyze the factors influencing insurance expenses.

The dataset includes the following features:

- **Age:** Age of the individual.
- **Sex:** Gender of the individual.
- **BMI:** Body Mass Index.
- **Children:** Number of dependent children.
- **Smoker:** Indicates whether the individual is a smoker.
- **Region:** Residential geographical region.
- **Charges:** Individual medical insurance expenses, which is the target variable.

The objective is to use these attributes to develop a machine learning model capable of predicting medical insurance charges.

Importing Libraries :

Several Python libraries are used throughout the project for data manipulation, visualization, preprocessing, model development, and evaluation.

- **Pandas:** For loading and manipulating dataset records.
- **NumPy:** For numerical operations.
- **Matplotlib:** For creating data visualizations.
- **Seaborn:** For statistical data visualization.
- **Scikit-learn:** For data preprocessing, regression algorithms, model training, and evaluation.
- **Joblib:** For saving and loading the trained machine learning model.

Code for importing libraries :

[117]:
```
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer

from sklearn.linear_model import LinearRegression, Lasso, Ridge

from sklearn.tree import DecisionTreeRegressor

from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

from sklearn.model_selection import GridSearchCV
```

Exploratory Data Analysis (EDA)

1. **Initial Data Inspection :**
   - Exploratory Data Analysis was performed to understand the dataset's structure, identify missing values and duplicate records, and examine the distribution of different features. Basic Pandas functions were used to inspect the dataset and understand its statistical properties.

Code for Data Overview :

```
[2]: # Load the data
insurance_data = pd.read_csv("../dataset/insurance.csv")
df = insurance_data.copy()
```

```
[3]: df.head()
```

[3]:

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

```
[5]: df.shape
```

[5]: (1338, 7)

```
[6]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1338 non-null   int64
 1   sex       1338 non-null   object
 2   bmi       1338 non-null   float64
 3   children  1338 non-null   int64
 4   smoker    1338 non-null   object
 5   region    1338 non-null   object
 6   charges   1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
```

```
[7]: df.describe()
```

[7]:

	age	bmi	children	charges
count	1338.000000	1338.000000	1338.000000	1338.000000
mean	39.207025	30.663397	1.094918	13270.422265
std	14.049960	6.098187	1.205493	12110.011237
min	18.000000	15.960000	0.000000	1121.873900
25%	27.000000	26.296250	0.000000	4740.287150
50%	39.000000	30.400000	1.000000	9382.033000
75%	51.000000	34.693750	2.000000	16639.912515
max	64.000000	53.130000	5.000000	63770.428010

```
[14]: df.isnull().sum()
```

[14]:

```
age         0
sex         0
bmi         0
children    0
smoker      0
region      0
charges     0
dtype: int64
```

```
[15]: df.duplicated().sum()
```

[15]: np.int64(1)

```
[16]: df[df.duplicated()]
```

[16]:

	age	sex	bmi	children	smoker	region	charges
581	19	male	30.59	0	no	northwest	1639.5631

2. Univariate Analysis :

- Univariate analysis was performed to understand the individual distribution of numerical and categorical features. Histograms were used to examine numerical attributes, while count plots were used to visualize the frequency of categorical values.

Code for univariate analysis :

```
[29]: # Distribution of Gender
df.groupby("sex")["charges"].agg(["count", "mean", "median"])
```

[29]:

sex	count	mean	median
female	662	12569.578844	9412.9625
male	675	13974.998864	9377.9047

```
[30]: # Distribution of somker
df.groupby("smoker")["charges"].agg(["count", "mean", "median"])
```

[30]:

smoker	count	mean	median
no	1063	8440.660307	7345.72660
yes	274	32050.231832	34456.34845

```
[31]: # Distribution by Region
df.groupby("region")["charges"].agg(["count", "mean", "median"])
```

[31]:

region	count	mean	median
northeast	324	13406.384516	10057.652025
northwest	324	12450.840844	8976.977250
southeast	364	14735.411438	9294.131950
southwest	325	12346.937377	8798.593000

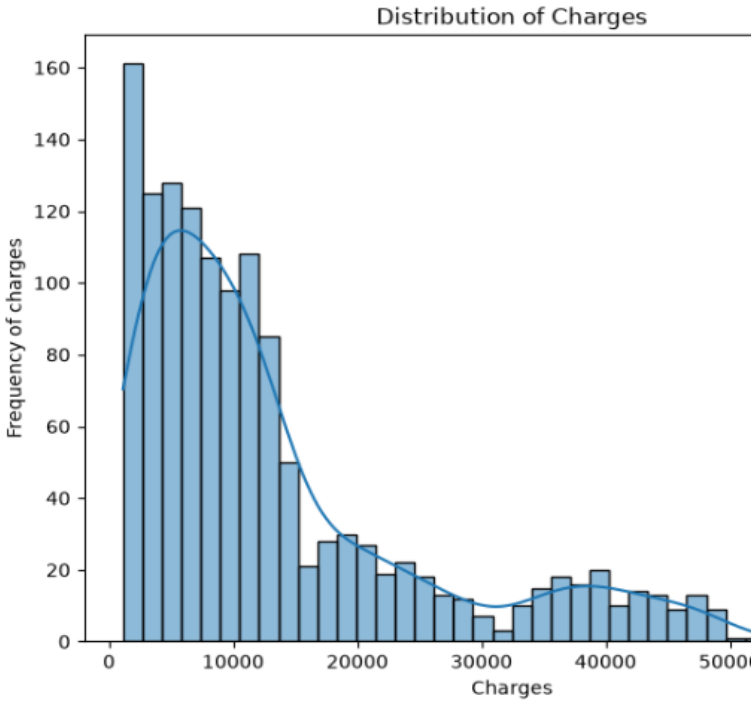
Distribution of Target variables

[23]:
```
fig, ax = plt.subplots(figsize = (8,6))

sns.histplot(data = df, x = "charges", bins = 40, kde = True, ax = ax)

ax.set_title("Distribution of Charges")
ax.set_xlabel("Charges")
ax.set_ylabel("Frequency of charges")

plt.show()
```

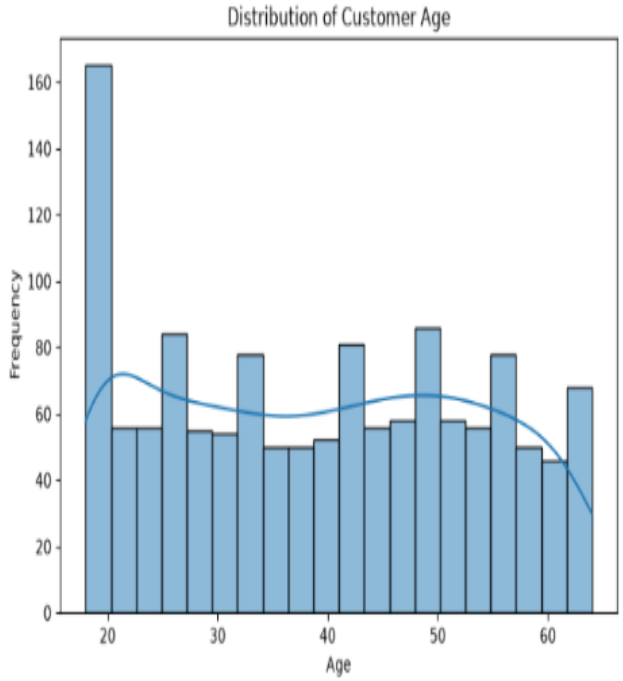


[26]:
```
# Distribution of Age
plt.figure(figsize=(8, 5))

sns.histplot(df['age'], bins=20, kde=True)

plt.title("Distribution of Customer Age")
plt.xlabel("Age")
plt.ylabel("Frequency")

plt.show()
```

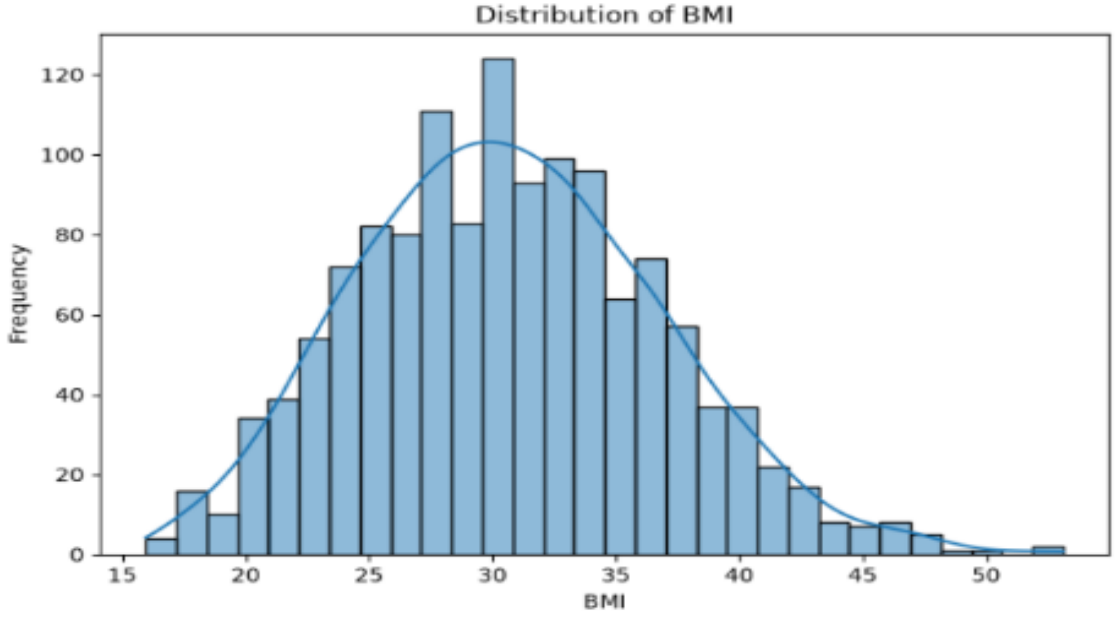


[27]:
```
# Distribution of BMI
plt.figure(figsize=(8, 5))

sns.histplot(df['bmi'], bins=30, kde=True)

plt.title("Distribution of BMI")
plt.xlabel("BMI")
plt.ylabel("Frequency")

plt.show()
```

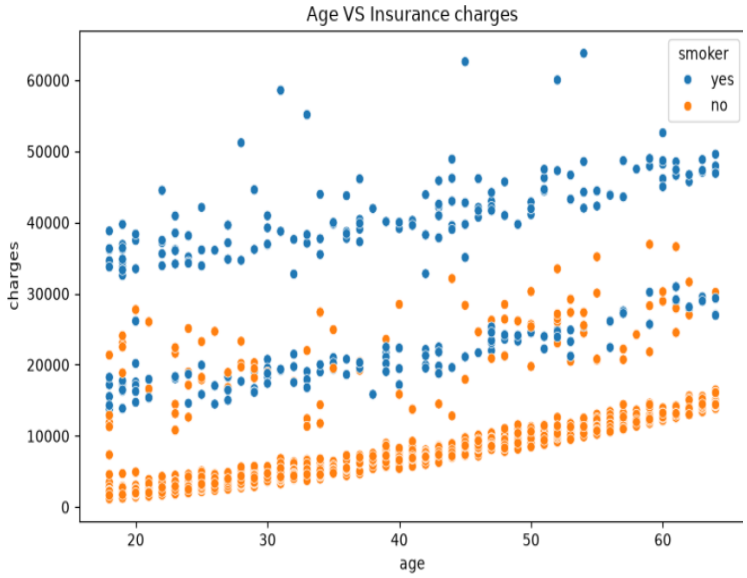


3. Bivariate Analysis :

- Bivariate analysis was conducted to examine the relationship between individual features and medical insurance charges. Scatter plots and box plots were used to identify patterns and understand how factors such as age, BMI, and smoking habits influence insurance expenses.

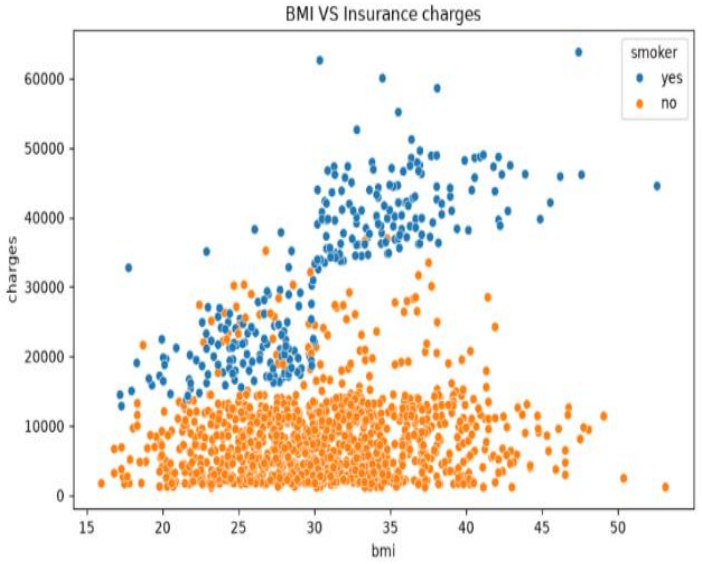
Age vs Insurance charge

```
[37]: plt.figure(figsize=(8, 5))
      sns.scatterplot(data = df, x = "age", y = "charges", hue = "smoker")
      plt.title("Age VS Insurance charges")
      plt.tight_layout()
      plt.show()
```



BMI vs Insurance charge

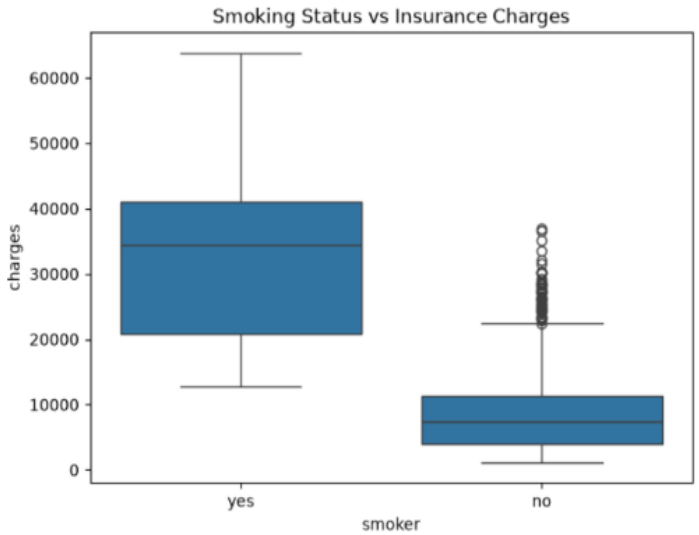
```
[38]: plt.figure(figsize=(8, 5))
      sns.scatterplot(data = df, x = "bmi", y = "charges", hue = "smoker")
      plt.title("BMI VS Insurance charges")
      plt.tight_layout()
      plt.show()
```



Smoker vs Insurance charges

```
[39]: plt.figure(figsize=(7, 5))

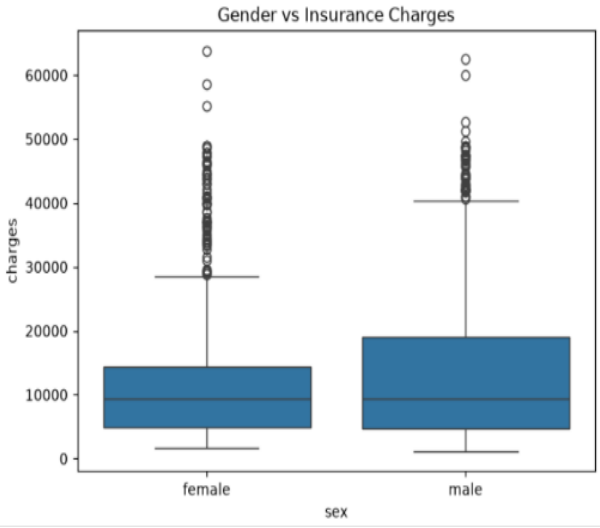
      sns.boxplot(
          data=df,
          x='smoker',
          y='charges'
      )
      plt.title("Smoking Status vs Insurance Charges")
      plt.show()
```



Gender vs Insurance charges

```
[40]: plt.figure(figsize=(7, 5))

      sns.boxplot(
          data=df,
          x='sex',
          y='charges'
      )
      plt.title("Gender vs Insurance Charges")
      plt.show()
```



4. Data Preprocessing and Feature Engineering :

- **Data Cleaning :**

  The dataset was examined for missing values and duplicate records. Necessary data-cleaning operations were performed to ensure that the dataset was suitable for further analysis and model development.

  There is no missing values in any column in the Dataset.

  One duplicated row is detected and I remove it.

```
[15]: df.duplicated().sum()
```

```
[15]: np.int64(1)
```

```
[16]: df[df.duplicated()]
```

[16]:

	age	sex	bmi	children	smoker	region	charges
581	19	male	30.59	0	no	northwest	1639.5631

```
[22]: # Drop duplicate rows
      df.drop_duplicates().reset_index(drop = True, inplace = True)
```

- **Feature Engineering :**

  Feature engineering was performed to create additional attributes that could help the model understand the combined influence of certain factors on insurance charges.

  Two interaction features were created:

  - **Age-Smoker:** Represents the interaction between an individual's age and smoking status.
  - **BMI-Smoker:** Represents the interaction between BMI and smoking status.

  These features were introduced to help the regression models capture potential combined effects of age and BMI with smoking habits.

  Code :

```
[78]: df["smoker_yes"] = df["smoker"].map({"yes" : 1, "no" : 0})
```

```
[83]: df["age_smoker"] = df["age"] * df["smoker_yes"]
      df["bmi_smoker"] = df["bmi"] * df["smoker_yes"]
```

- **Encoding and Scaling :**

  Since the dataset contains both numerical and categorical features, separate preprocessing techniques were applied. StandardScaler was used to scale numerical features, while OneHotEncoder was used to convert categorical features into numerical representations. Missing values were handled using appropriate imputation strategies within the preprocessing pipeline.

  Code :

```
[96]: numerical_preprocessor = Pipeline([
          ("impute", SimpleImputer(strategy="median")),
          ("scaler", StandardScaler())
      ])

      categorical_preprocessor = Pipeline([
          ("impute", SimpleImputer(strategy="most_frequent")),
          ("encoder", OneHotEncoder(handle_unknown = "ignore"))
      ])
```

```
[97]: preprocessor = ColumnTransformer([
          ("numerical", numerical_preprocessor, numerical_features),
          ("categorical", categorical_preprocessor, categorical_features)
      ])
```

- **Train-Test Split :**

  The processed dataset was divided into training and testing subsets using an 80:20 ratio. The training data was used to train the machine learning models, while the testing data was reserved to evaluate their performance on unseen records. A random state of 42 was used to maintain reproducibility.

  Code :

```
[95]: X_train, X_test, y_train, y_test = train_test_split(
          X, y,
          test_size = 0.2,
          random_state = 42
      )
```

5. Model Implementation and Training :

- **Machine Learning Algorithms :**

  Five supervised machine learning regression algorithms were implemented to predict medical insurance charges:

  1. **Linear Regression:** Establishes a linear relationship between input features and the target variable.
  2. **Ridge Regression:** Uses L2 regularization to reduce the impact of large model coefficients.
  3. **Lasso Regression:** Uses L1 regularization and can reduce less important feature coefficients.
  4. **Decision Tree Regression:** Predicts insurance charges using a tree-based decision-making structure.
  5. **Random Forest Regression:** Combines multiple decision trees to produce more stable predictions.

  Each algorithm was trained using the prepared training dataset.

- **Model Training :**

  A preprocessing and model-training pipeline was created using Scikit-learn. Each regression algorithm was combined with the preprocessing pipeline to ensure consistent data transformation during training and prediction.The five models were trained using the training dataset. Their initial performance was then examined to understand how effectively each algorithm could predict medical insurance charges.

Code:

```
[100]: model_pipelines = {}

for name, model in models.items():
    pipeline = Pipeline([
        ("Preprocessor", preprocessor),
        ("model", model)
    ])
    model_pipelines[name] = pipeline
```

Train the 5 Regressor model

```
[101]: for name, pipeline in model_pipelines.items():
           pipeline.fit(X_train, y_train)

           print(f"{name} fitted successfully")
```

```
Linear Regression fitted successfully
Ridge Regression fitted successfully
Lasso Regression fitted successfully
Decision Tree fitted successfully
Random Forest fitted successfully
```

```
[105]: results_df = pd.DataFrame(results)

results_df = results_df.sort_values(
    by='RMSE',
    ascending=True
)

results_df
```

[105]:

	Model	MAE	RMSE	R2 Score
4	Random Forest	2725.591570	4884.659511	0.821925
2	Lasso Regression	2993.687177	4952.701796	0.818234
0	Linear Regression	2993.720836	4952.728043	0.818232
1	Ridge Regression	2996.121674	4957.068490	0.817981
3	Decision Tree	3091.130047	6472.547097	0.689223

6. Hyperparameter Tuning and Model Evaluation :

- **Hyperparameter Tuning**

  Hyperparameter tuning was performed to identify suitable parameter combinations and improve the performance of the regression models. GridSearchCV was used with 5-fold cross-validation to evaluate different parameter combinations.

  The tuning process was applied to Ridge Regression, Lasso Regression, Decision Tree Regression, and Random Forest Regression. Linear Regression was retained as a baseline model for comparison.

Code:

Random Forest Regressor

[130]:
```
rf_pipeline = Pipeline(
    steps=[
        ('preprocessor', preprocessor),
        ('model', RandomForestRegressor(random_state=42))
    ]
)

rf_params = {
    'model__n_estimators': [100, 200],

    'model__max_depth': [5, 10, None],

    'model__min_samples_split': [2, 5],

    'model__min_samples_leaf': [1, 2]
}

rf_grid = GridSearchCV(
    rf_pipeline,
    rf_params,
    cv=5,
    scoring='neg_root_mean_squared_error',
    n_jobs=-1
)

rf_grid.fit(X_train, y_train)
```

Decision Tree Regressor

[124]:
```
dt_pipeline = Pipeline(
    steps=[
        ('preprocessor', preprocessor),
        ('model', DecisionTreeRegressor(random_state=42))
    ]
)

dt_params = {
    'model__max_depth': [3, 5, 10, None],

    'model__min_samples_split': [2, 5, 10],

    'model__min_samples_leaf': [1, 2, 4]
}

dt_grid = GridSearchCV(
    dt_pipeline,
    dt_params,
    cv=5,
    scoring='neg_root_mean_squared_error',
    n_jobs=-1
)

dt_grid.fit(X_train, y_train)
```

Lasso Regression

[121]:
```
lasso_pipeline = Pipeline(
    steps=[
        ('preprocessor', preprocessor),
        ('model', Lasso(max_iter=20000))
    ]
)

lasso_params = {
    'model__alpha': [0.001, 0.01, 0.1, 1, 10]
}

lasso_grid = GridSearchCV(
    lasso_pipeline,
    lasso_params,
    cv=5,
    scoring='neg_root_mean_squared_error',
    n_jobs=-1
)

lasso_grid.fit(X_train, y_train)
```

Ridge Regression

[118]:
```
ridge_pipeline = Pipeline(
    steps=[
        ('preprocessor', preprocessor),
        ('model', Ridge())
    ]
)

ridge_params = {
    'model__alpha': [0.01, 0.1, 1, 10, 100]
}

ridge_grid = GridSearchCV(
    ridge_pipeline,
    ridge_params,
    cv=5,
    scoring='neg_root_mean_squared_error',
    n_jobs=-1
)

ridge_grid.fit(X_train, y_train)
```

- **Model Evaluation:**

The trained models were evaluated using the testing dataset to measure their prediction performance. Three evaluation metrics were used:

- **Mean Absolute Error (MAE):** Measures the average absolute difference between actual and predicted charges.
- **Root Mean Squared Error (RMSE):** Measures prediction errors while giving greater weight to larger errors.
- **R-squared (R^2):** Indicates how well the model explains the variation in insurance charges.

These metrics were used to compare the models and identify the most suitable algorithm for the final prediction system.

Code:

```
[136]: final_results_df = pd.DataFrame(final_results)

final_results_df.sort_values(
    by='RMSE',
    ascending=True
)
```

[136]:

	Model	MAE	RMSE	R2 Score
4	Random Forest Tuned	2422.166935	4262.798957	0.901111
3	Decision Tree Tuned	2681.721070	4416.820776	0.893836
1	Ridge Tuned	2831.466334	4576.139748	0.886039
2	Lasso Tuned	2831.308617	4576.630635	0.886014
0	Linear Regression	2831.646978	4577.779551	0.885957

- **Final Model Selection :**

Based on the evaluation results, the most suitable regression model was selected by considering its MAE, RMSE, and R^2 scores. The selected model was saved using Joblib so that it could be integrated into the Streamlit web application for predicting medical insurance charges.

Code :

```
[137]: best_model_name = final_results_df.sort_values(
    by='RMSE',
    ascending=True
).iloc[0]['Model']

final_model = final_models[best_model_name]

print("Best Model:", best_model_name)
```

```
Best Model: Random Forest Tuned
```

```
[139]: import os
import joblib
os.makedirs("../models", exist_ok=True)

joblib.dump(
    final_model,
    "../models/final_insurance_model.pkl"
)

print("Final model saved successfully!")
```

```
Final model saved successfully!
```

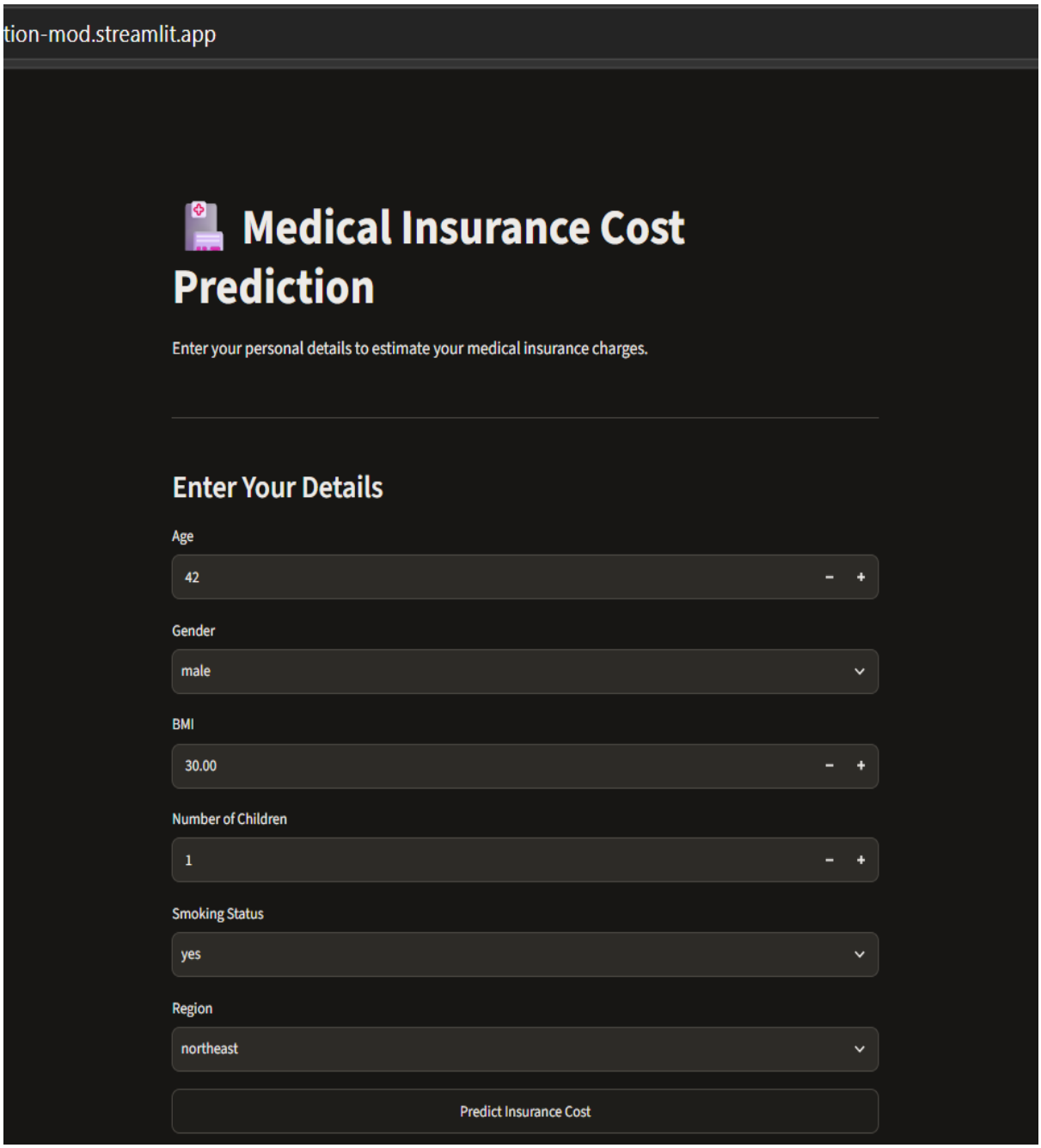
Web Application and Interface

1. **Web Application Development**

   The web application is developed using Streamlit, which provides a simple interface for interacting with the trained machine learning model. Users can enter their personal details, including age, gender, BMI, number of children, smoking status, and region, to estimate their medical insurance charges.

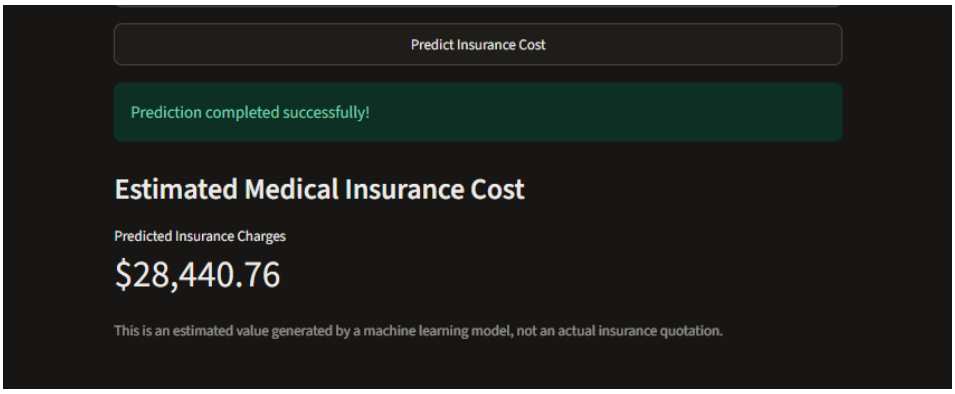
2. **User Interface**

   The application provides input fields and selection options to collect the required information from users.



3. Prediction Results

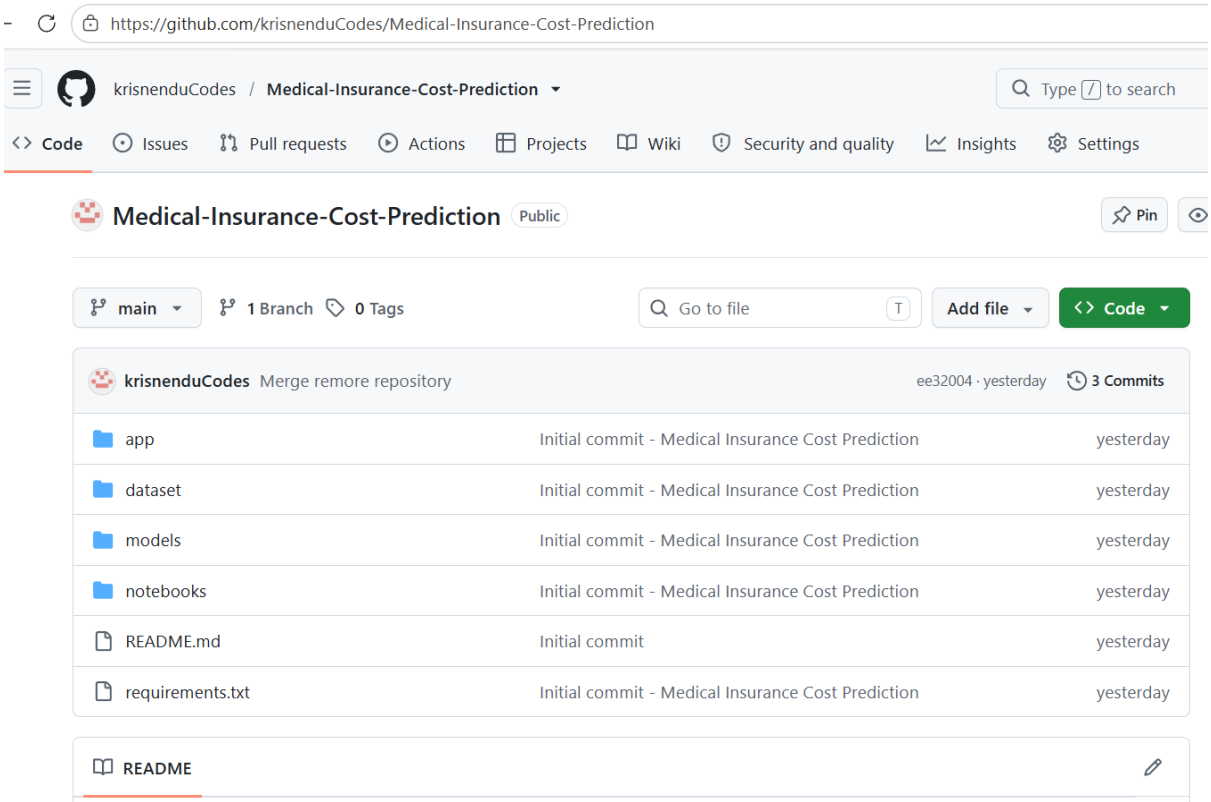
After submitting the details, the application processes the input through the trained machine learning pipeline and displays the predicted medical insurance charges in US dollars.



DEPLOYMENT

1. GitHub Repository

The complete project, including the source code, dataset, trained model, and required libraries, is uploaded to GitHub for project storage and version management.



2. Application Deployment

The Streamlit application is deployed using Streamlit Community Cloud by connecting the GitHub repository. After successful deployment, a public URL is generated, allowing users to access the application through a web browser.

File Structure :

```
Medical-Insurance-Cost-Prediction/
│
├── dataset/
│       └── insurance.csv
│
├── notebooks/
│       └── medical_insurance_cost_prediction.ipynb
│
├── models/
│       └── final_insurance_model.pkl
│
├── app/
│       └── app.py
│
├── requirements.txt
│
└── README.md
```

Live Application Link: [krisnenduCodes/Medical-Insurance-Cost-Prediction: Medical Insurance cost prediction using supervised learning]

CONCLUSION

The **Medical Insurance Cost Prediction Using Supervised Machine Learning** project demonstrates the practical application of machine learning techniques to estimate individual medical insurance expenses based on personal and lifestyle-related factors.

Using the Medical Cost Personal Dataset, the project performs data analysis, preprocessing, and feature engineering to prepare the data for model development. Five regression algorithms—Linear Regression, Ridge Regression, Lasso Regression, Decision Tree Regression, and Random Forest Regression—are implemented and compared using MAE, RMSE, and R^2 metrics. Hyperparameter tuning is also performed to improve model performance and identify the most suitable prediction model.

The project further integrates the trained machine learning pipeline into a Streamlit web application, allowing users to enter their personal details and receive an estimated medical insurance cost through a simple interface.

The system also provides a foundation for future improvements, such as using larger and more diverse datasets, incorporating additional health-related factors, improving prediction accuracy, and developing more advanced user interfaces.

Overall, this project demonstrates a complete machine learning workflow, from dataset exploration and model development to evaluation, application integration, and online deployment.